Cryptography for Enterprises: Verifiable Credentials

Privacy-preserving trust for digital interactions

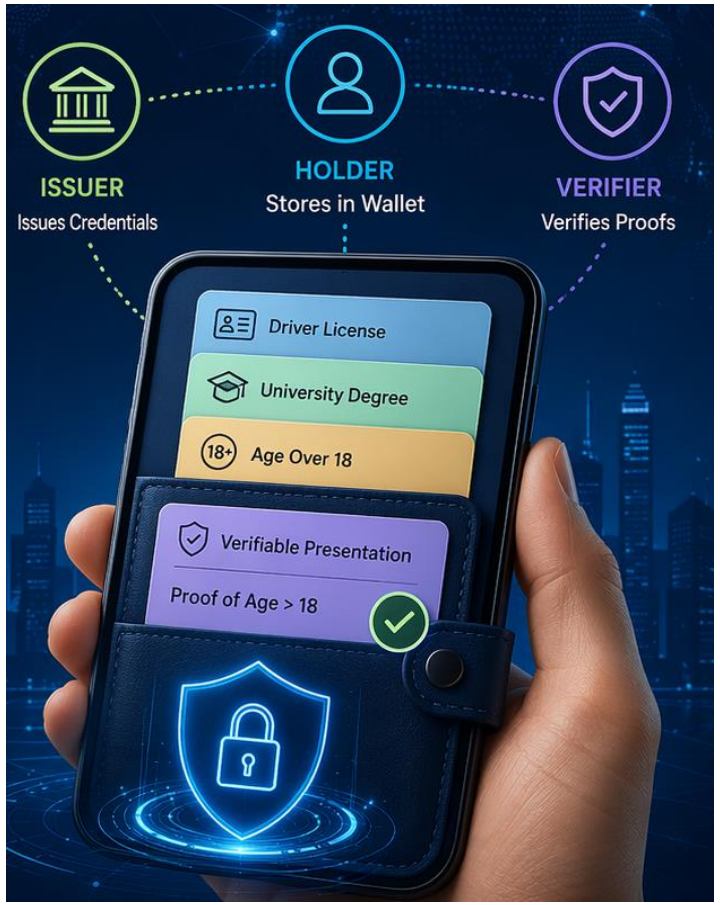
Vigneswaran R

Senior Scientist, TCS Research

ACM Summer School 2026

16 June 2026

vigneswaran.r@tcs.com



Note: AI-assisted drafting and review; final content verified by author.

Agenda

1. Why do digital credentials matter?
2. Why are current identity systems not enough?
3. What are VCs, VPs, wallets, DIDs and VDRs?
4. What privacy/security properties do we want?
5. How do signatures, ZK proofs and accumulators help?

Why credentials matter

Credentials answer: Who are you? What do you own? What are you allowed to do?

Human credentials

KYC / onboarding

Driving licenses, degrees, permits

Access control and accountability

Healthcare, benefits, age-restricted services

Non-human credentials

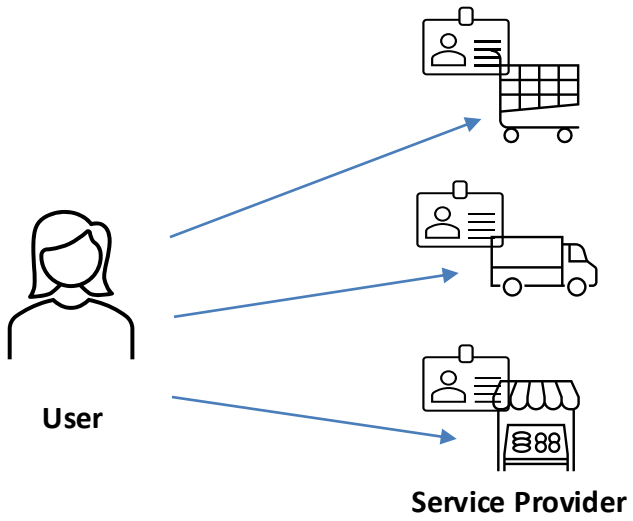
KYA: agents and bots

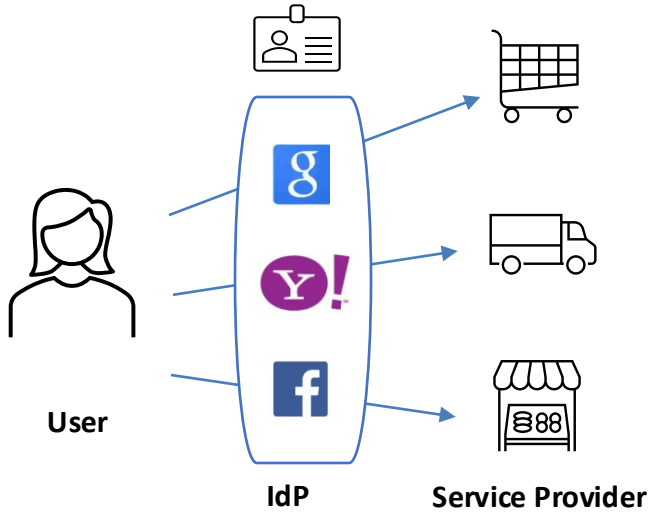
Device / workload identity

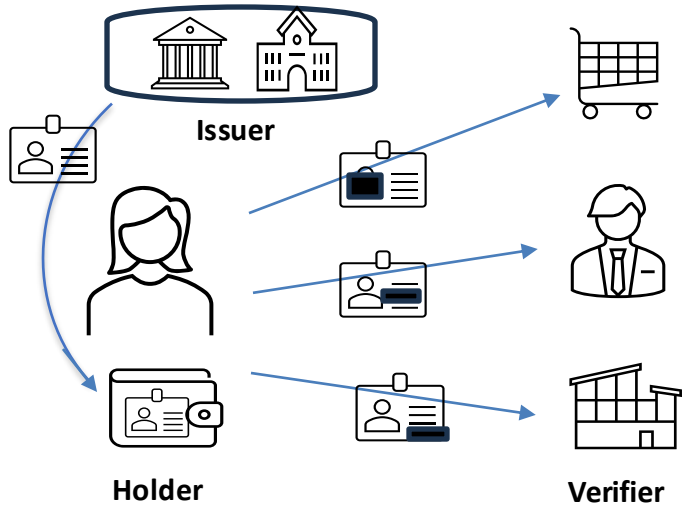
Battery passports

Track-and-trace and supply chains

Identity evolution







Centralised

Identity fragmented across services

Multiple usernames / passwords

Poor UX, weak privacy

Federated

Identity moved to IdP

Single sign-on improves UX

Still provider-centric

Decentralised

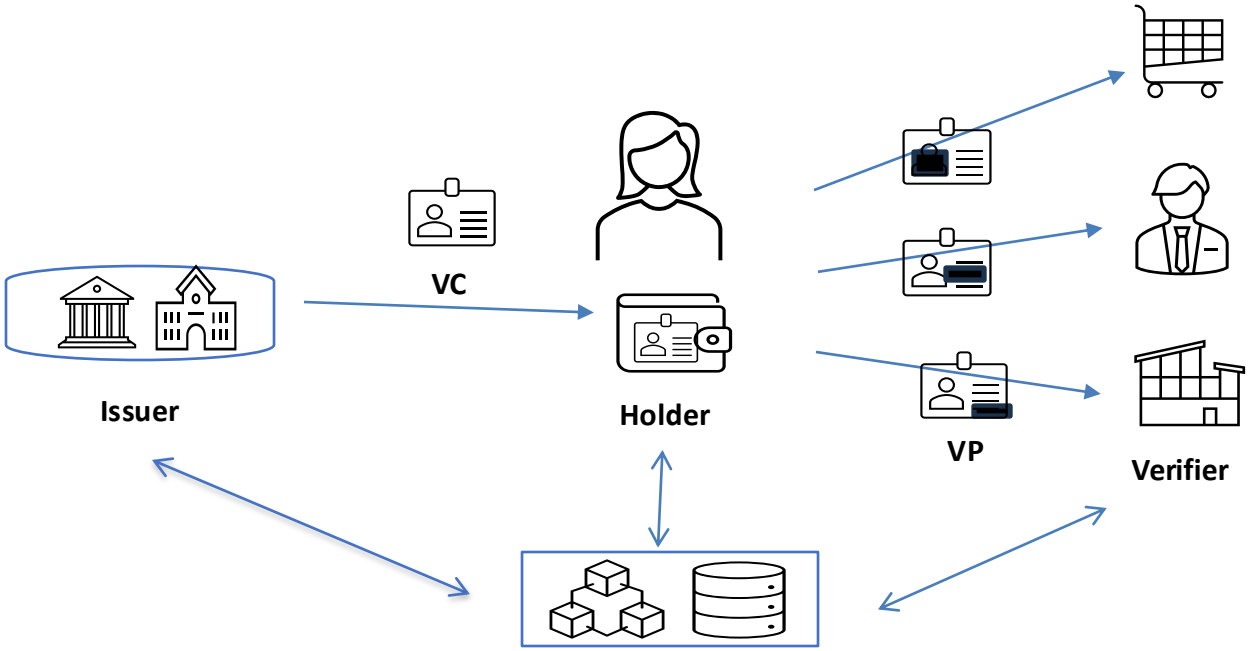
Credentials held by user

Selective disclosure

Verifier need not contact issuer

Control of data shifts: Service Provider side → Identity Provider side → Holder side

Decentralised identity: closer look



VC: Verifiable Credential

VP: Verifiable Presentation

Wallet: Stores VC(s) and creates VP(s)

VDR: Stores keys, schemas, revocation metadata

DID: Decentralized identifier – resolves to public keys /metadata

1. Issue
Issuer signs attributes and gives VC to holder

→

2. Store
Holder stores VC in Wallet

→

3. Present
Holder derives VP (selective claims / proofs)

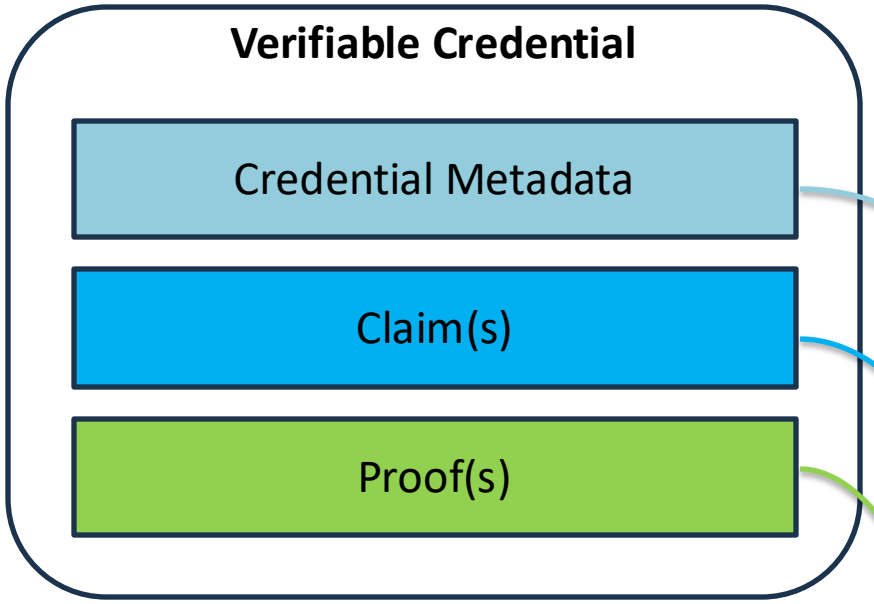
→

4. Verify
Verifier checks signature, keys and revocation data

Verification can happen without contacting the issuer during every presentation.

DID Image Source: https://www.w3.org/TR/did-1.0/

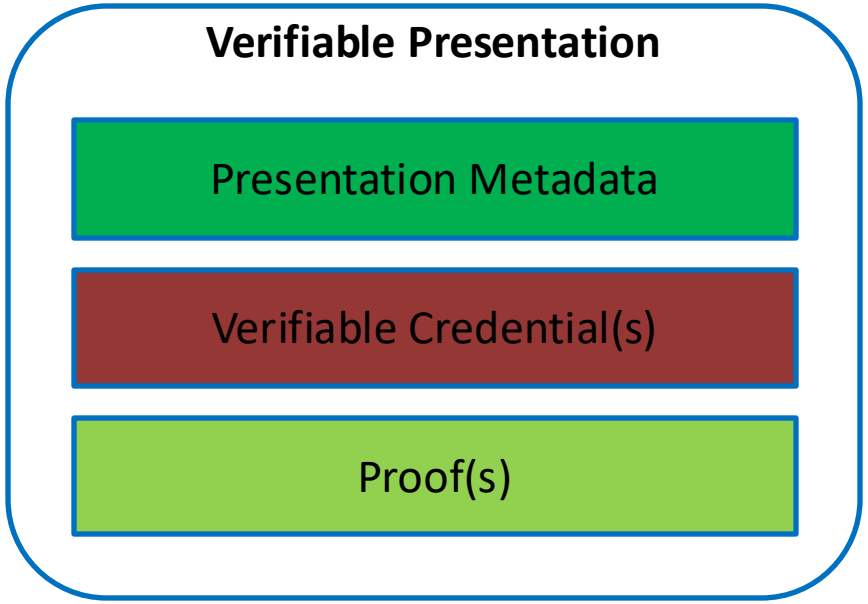
Verifiable Credential Structure

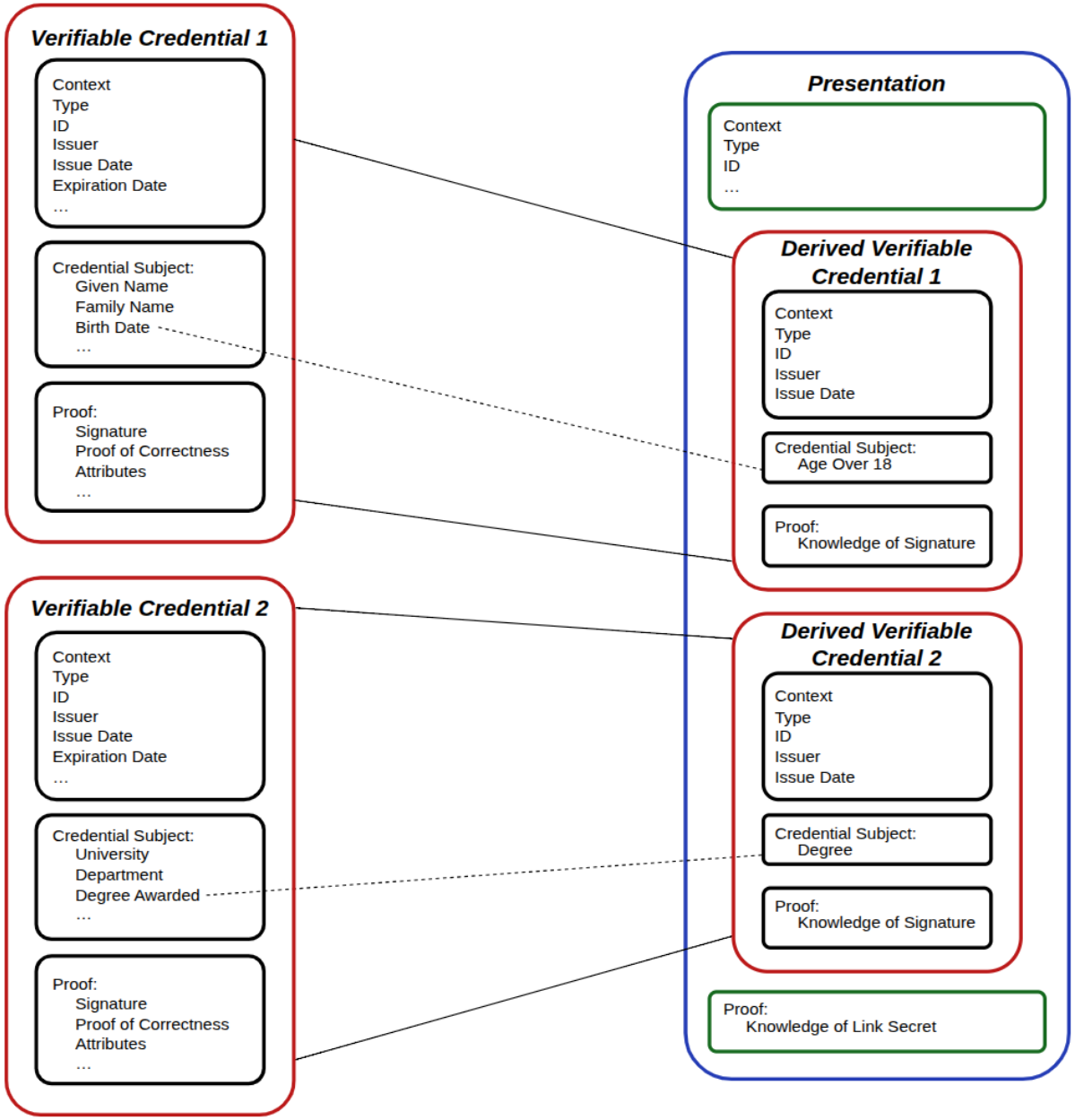



```
"@context": [
  "https://www.w3.org/ns/credentials/v2",
  "https://www.w3.org/ns/credentials/examples/v2"
],
"id": "http://university.example/credentials/3732",
"type": [
  "VerifiableCredential",
  "ExampleDegreeCredential"
],
"issuer": {
  "id": "did:example:76e12ec712ebc6f1c221ebfeb1f",
  "name": "Example University"
},
"validFrom": "2010-01-01T19:23:24Z",
"credentialSubject": {
  "id": "did:example:ebfeb1f712ebc6f1c276e12ec21",
  "degree": {
    "type": "ExampleBachelorDegree",
    "name": "Bachelor of Science and Arts"
  }
},
"proof": {
  "type": "DataIntegrityProof",
  "created": "2025-04-27T17:58:33Z",
  "verificationMethod": "did:key:zDnaebSRtPnW6YCpxAhR5JPx.
  "cryptosuite": "ecdsa-rdfc-2019",
  "proofPurpose": "assertionMethod",
  "proofValue":
"z35CwmxThsUQ4t79JfacmMcw4y1kCqtD4rKqUooKM2NyKwdF5jmXMRo9oG
4"
```

Source: https://www.w3.org/TR/vc-data-model-2.0/

Verifiable Presentation Structure





Source: https://www.w3.org/TR/vc-data-model-2.0/

From older credentials to modern requirements

Older credential formats

Physical cards / paper: theft, loss, wear-and-tear

Signed PDF: not natively machine-readable

Often full disclosure of all data – burden/risk on data custodian

Hard to automate and hard to minimise data shared

Modern VC requirements

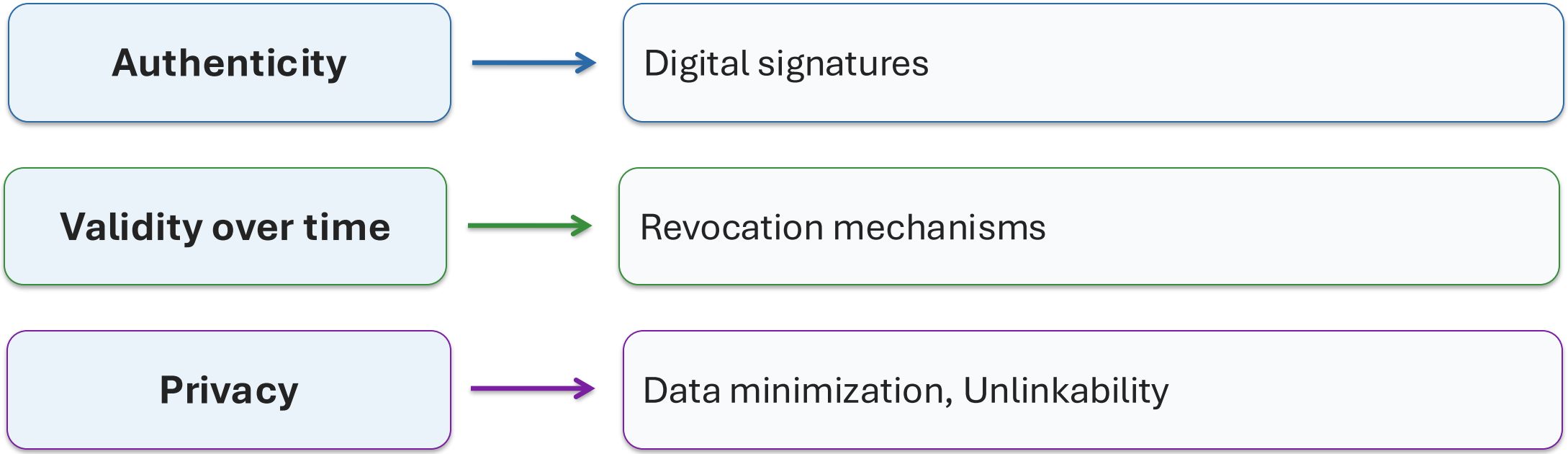
Machine readability and verifiability

Selective disclosure: reveal only what is needed

Predicate proofs: e.g. “Age > 18” without DoB

Unlinkability: repeated uses of the same credential should not be correlated

Three cryptographic goals



We now look at three families: SD-JWT, ZK-friendly signatures, and cryptographic accumulators.

SD-JWT Sample Presentation

Token:


```
{
  "alg": "RS256",
  "kid": "uidai-37a8e742-c375-443c-8ce5-8b7a0822cb6b",
  "typ": "dc+sd-jwt"
}
{
  "iss": "https://pehchaan.uidai.gov.in",
  "exp": 1813036111,
  "iat": 1781500111,
  "id": "a0e3b173-bd03-b91b-ac34-1a3c74fdfcc2",
  "cnf": {
    "jwk": {
      "e": "KAGQ",
      "kty": "RSA",
      "n": "VaA4XaTxQWyccu5fDxOkc8M0Ara4QeknOjhurx..."
    }
  },
  "_sd_alg": "sha-256",
  "_sd": [
    ...
    "-2Sb9DxvzgjQL1PUEeQsfYZrmEjTJVmb7tn_qjQZtdE",
    "sA54ups7FA6_c29pOpH5mDCcxJh4_dTPgHo3-cxsVkc",
    "sBh8iQR3z6h11YrFYH6KWKa7a2vCH0ZI18CPG_1_IPA",
    "_jjECteE6jEK5Sj0Inah2BFHEyt0w8pHjZw1tt9dG9w",
    ...
  ],
  "vct": "http://pehchaan.uidai.gov.in/credentials/
                    VerifiedAadhaarCredential/v1"
}
...
```


Format:

```
<Issuer-signed JWT>~<Disclosure 1>~<Disclosure 2>~...~<Disclosure N>~

JWT = base64url(header) . base64url(payload) . base64url(signature)
```

Disclosures:


```
["ede70dce-2112-4330-9ae0-27bec356b69f","ResidentName","R Vigneswaran"]
["9f74c93a-a3f3-4631-80ed-9aec698f55e5","AgeAbove18","Yes"]
```


Format: bse64url(hash(base64url(JSON([salt, claim, value]))))


```
-2Sb9DxvzgjQL1PUEeQsfYZrmEjTJVmb7tn_qjQZtdE
sBh8iQR3z6h11YrFYH6KWKa7a2vCH0ZI18CPG_1_IPA
```

SD-JWT Sample Presentation

Token:

```
{
  "alg": "ES256",
  "kid": "...e5-8b7a0822cb6b",
  "...": "..."
}
{
  "...": "...",
  "...": "...",
  "...": "...4fdfcc2...",
  "...": "...",
  "...": "...a4QeknOjhurx..."
  }
  "...": "...",
  "...": "...",
    "...": "...tn_qjOZtdE",
    "...": "...Ho3-cxsVkc",
    "...": "...8CPG_1_IPA",
    "...": "...Zw1tt9dG9w",
  ],
  "vct": "http://pehchaan.uidai.gov.in/credentials/
                    VerifiedAadhaarCredential/v1"
}
...
```

Issuer-signed JWT

- clear claims + _sd hashes
- issuer signature on the JWT
- optional holder binding (cnf / KB-JWT)

Selective disclosures [salt, claim name, value]

Format:

```
<Issuer-signed JWT>~<Disclosure 1>~<Disclosure 2>~...~<Disclosure N>~
```

Verifier reconstructs and checks against signed hashes

Disclosures:

```
["ede70dce-2112-4330-9ae0-27bec356b69f","ResidentName","R Vigneswaran"]
["9f74c93a-a3f3-4631-80ed-9aec698f55e5","AgeAbove18","Yes"]
```

Format: bse64url(hash(base64url(JSON([salt, claim, value]))))

```
-2Sb9DxvzgjQL1PUEeQsfYZrmEjTJVmb7tn_qjOZtdE
sBh8iQR3z6h11YrFYH6KWKa7a2vCH0ZI18CPG_1_IPA
```

SD-JWT Sample Presentation

Token:

```
{
  "alg": "RS256",
  "kid": "uidai-37a8e742-c375-443c-8ce5-8b7a0822cb6b",
  "typ": "dc+sd-jwt"
}
{
  "iss": "https://pehchaan.uidai.gov.in",
  "exp": 1813036111,
  "iat": 1781500111,
  "id": "a0e3b173-bd03-b91b-ac34-1a3c74fdfcc2",
  "cnf": {
    "jwk": {
      "e": "KAGQ",
      "kty": "RSA",
      "n": "VaA4XaTxQWyccu
    }
  },
  "_sd_alg": "sha-256",
  "_sd": [
    ...
    "-2Sb9DxvzgjQL1PUEeQsfYZrmEjTJVmb7tn_qjQZtdE",
    "sA54ups7FA6_c29pOpH5mDCcxJh4_dTPgHo3-cxsVkc",
    "sBh8iQR3z6h11YrFYH6KWKa7a2vCH0ZI18CPG_1_IPA",
    "_jjECteE6jEK5Sj0Inah2BFHEyt0w8pHjZw1tt9dG9w",
    ...
  ],
  "vct": "http://pehchaan.uidai.gov.in/credentials/
                    VerifiedAadhaarCredential/v1"
}
...
```

Format:

```
<Issuer-signed JWT>~<Disclosure 1>~<Disclosure 2>~...~<Disclosure N>~

JWT = base64url(header) . base64url(payload) . base64url(signature)
```

Pros: simple, practical deployment
Limits: selective disclosure only; no native predicates, unlinkability

Disclosures:

```
["ede70dce-2112-4330-9ae0-27bec356b69f","ResidentName","R Vigneswaran"]
["9f74c93a-a3f3-4631-80ed-9aec698f55e5","AgeAbove18","Yes"]
```

Format: `bse64url(hash(base64url(JSON([salt, claim, value]))))`

```
-2Sb9DxvzgjQL1PUEeQsfYZrmEjTJVmb7tn_qjQZtdE
sBh8iQR3z6h11YrFYH6KWKa7a2vCH0ZI18CPG_1_IPA
```

Understanding BBS+ Signatures and Their Role in Privacy-Preserving Credentials

Features

- Signs multiple attributes together
- Selective disclosure of chosen attributes
- Unlinkable proofs across presentations
- Proof of possession (not raw signature)

BBS+ = selective disclosure + unlinkability over signed attributes

Messages: m1, m2, …, mn
Signature: σ = Sign(sk, m1, m2, …, mn)

σ proves:
"These messages were signed by issuer"

Reveal subset:
m2, m4

Hide:
m1, m3, m5

PoK:
"I know σ on messages {mi}
such that:
disclosed = {m2, m4}
hidden attributes remain committed"

Understanding BBS+ Signatures and Their Role in Privacy-Preserving Credentials

Features

- Signs multiple attributes together
- Selective disclosure of chosen attributes
- Unlinkable proofs across presentations
- Proof of possession (not raw signature)

Messages: m1, m2, …, mn
Signature: σ = Sign(sk, m1, m2, …, mn)

σ proves:
"These messages were signed by issuer"

Reveal subset:
m2, m4

Hide:

BBS+ = selective disclosure + unlinkability over s
attributes

Limitations

- Predicates require external ZK layer
- Higher computation and proof cost

hidden attributes remain committed"

Zero-knowledge proof: concept

Prove "I know x such that f(x) satisfies condition P" without revealing x.

Proof Goal	Secret (Prover)	Public (Verifier)
Factorization	(p, q)	N = pq
Hash pre-image	x	hash(x)
Commitment opening	(x, r)	C
Age ≥ 18	Age / DoB	Threshold
Not revoked	Credential ID	Accumulator
Discrete log	x	g^x

Completeness
Honest prover succeeds

Soundness
Cheating prover should fail

Zero-knowledge
Verifier learns nothing beyond validity

Zero-knowledge proof: example

Goal: Prove knowledge of x such that
$y = g^x \bmod p$
without revealing x

1. Prover chooses random r
   $t = g^r \bmod p$

2. Verifier sends challenge
   $c \in \{0,1\}$

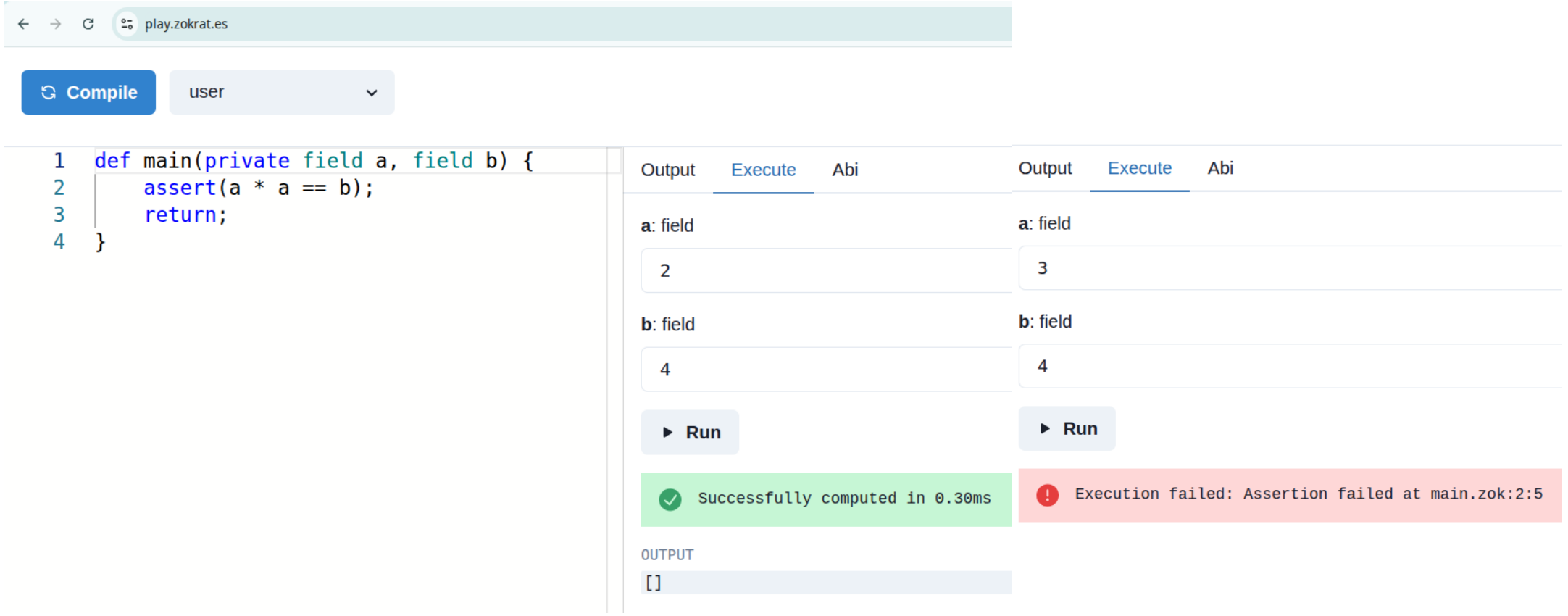
3. Prover computes
   $v = r + c \cdot x$

4. Prover sends v

5. Verifier checks:

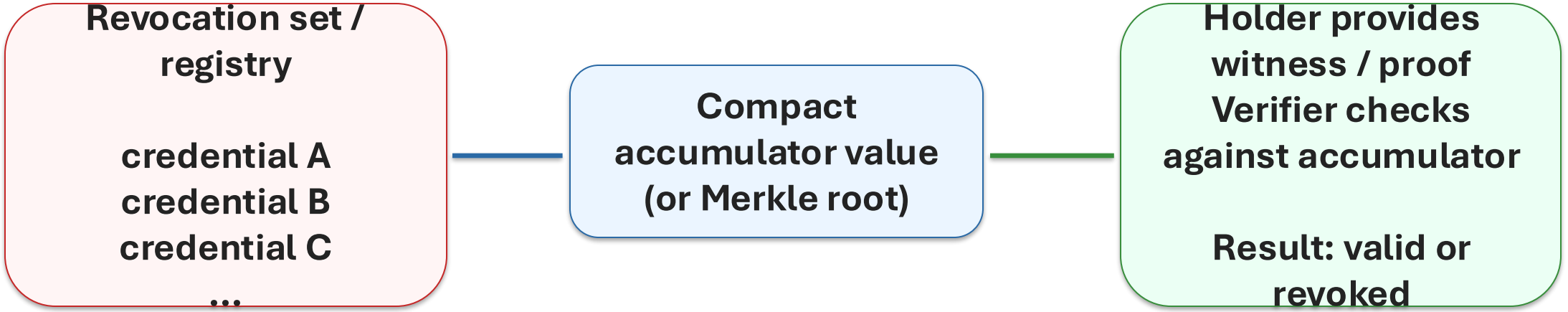
   $g^v \stackrel{?}{=} t \cdot y^c \bmod p$ $\qquad$ $g^{(r + c \cdot x)} = g^r \cdot (g^x)^c = t \cdot y^c$

Zero-knowledge proof: example



Cryptographic accumulators: intuition for revocation

Goal: check whether a credential is still valid, without sacrificing privacy or scalability.



Examples: Merkle trees, RSA accumulators, vector commitments

Cryptographic accumulators: intuition for revocation

Goal: check whether a credential is still valid, without sacrificing privacy or scalability.

Revocation set / registry

credential A

Compact accumulator value

Holder provides witness / proof
Verifier checks against accumulator

Toy accumulator example (intuition):
Revoked = {2, 5, 7}; Acc = 2 × 5 × 7 = 70
Credential ID = 3 ⇒ gcd(3, 70) = 1 ⇒ not revoked

Real system: holder sends a ZK membership / non-membership proof,
not the credential ID or the full revocation list.

Examples: Merkle trees, RSA accumulators, vector commitments

Summary of main approaches

Scheme	Primitive	ZK & privacy features	Key strength	Key limitation
SD-JWT	Hash + standard sigs	Sel. disclosure only No native predicates No inherent unlinkability	Simple and deployable	Limited privacy
BBS / BBS+	Pairing-based ECC	Sel. disclosure Predicates Unlinkable	Best balance	Limited native support for complex predicates
Generic ZK + sigs	Any sig + SNARK/MPC	Sel. disclosure Arbitrary predicates Unlinkability	Most expressive	High complexity

Design trade-off: SD-JWT (simple) ↔ BBS+/CL/PS (balanced) ↔ SNARKs (most expressive but heavy)

Key take aways

Modern credentials are not just signed documents — they are cryptographic tools for trustworthy automation with privacy.

- Identity evolved from provider-controlled to holder-controlled
- VCs separate issuance from presentation
- Selective disclosure and predicates reduce oversharing
- ZK-friendly signatures such as BBS+ make repeated use privacy-preserving
- Revocation is where scalability and privacy collide

Thank you

Questions welcome

Vigneswaran R
Senior Scientist, TCS Research
vigneswaran.r@tcs.com